OSLpay 商户入驻平台 — 用户手册

修订记录

版本	日期	修订内容
1.2	2024-03-04	新增 MCP 配置中心使用指南
1.1	2024-03-04	1. 新增快速开始页面使用指南 2. 新增菜单结构说明 3. 新增成员与权限管理使用指南 4. 新增域名白名单管理使用指南 5. 新增 API 请求日志查看使用指南 6. 完善 API 文档功能说明（16 个接口）
1.0	2024-03-03	初始版本

平台简介

OSLpay 商户入驻平台是一个面向商户的综合性门户，提供：

- **商户入驻申请**：在线提交入驻材料，追踪审核进度
- **业务数据查看**：实时查看交易数据、订单状态
- **API 文档浏览**：AI 友好的交互式 API 文档
- **开发工具**：签名验证、代码生成、Webhook 配置
- **双环境支持**：生产环境和沙箱环境独立管理

核心优势

1. **AI 驱动的开发体验** - 一键生成 AI 提示词，快速集成 API
2. **双环境隔离** - 沙箱环境安全测试，生产环境稳定运行
3. **角色权限管理** - 业务人员和技术人员各司其职
4. **实时通知** - Webhook 推送，及时获取业务事件

快速开始

访问平台

1. 打开浏览器，访问：`https://merchant.oslpay.com`（或您的部署地址）
2. 使用您的账号和密码登录
3. 首次登录后，系统会自动跳转到快速开始页面
4. 系统会根据您的角色展示相应的功能菜单

测试账号（演示环境）

账号	密码	角色	可访问功能
admin	demo123	管理员	全部功能
bd	demo123	商务人员	仪表盘、商户入驻申请
tech	demo123	技术人员	API 文档、开发者控制台、签名工具、Webhook 管理

环境切换

平台支持生产环境和沙箱环境：

- 点击顶部栏右上角的「生产 / 沙箱」按钮切换环境
- 沙箱环境用于测试，不产生实际费用
- 生产环境用于正式业务，需要完整的审核流程

角色与权限

管理员（Admin）

权限范围：全部功能

主要职责：

- 管理商户账户和用户
- 查看所有业务数据
- 配置系统参数
- 审核入驻申请

可访问菜单：

- 快速开始

- 仪表盘
- 业务管理
  - 入驻申请
  - 成员与权限
- 开发者套件
  - API 文档
  - API 凭证
  - 域名白名单
  - Webhook 管理
  - 签名工具
  - API 请求日志

商务人员（Business）

权限范围：业务相关功能

主要职责：

- 提交和跟踪入驻申请
- 查看交易数据和订单状态
- 导出业务报表

可访问菜单：

- 快速开始
- 仪表盘
- 业务管理
  - 入驻申请

技术人员（Technical）

权限范围：技术集成功能

主要职责：

- 浏览 API 文档
- 管理 API 密钥
- 配置 Webhook
- 调试签名和加密

可访问菜单：

- 快速开始
- 开发者套件
  - API 文档
  - API 凭证
  - 域名白名单
  - Webhook 管理
  - 签名工具
  - API 请求日志

功能模块

快速开始页面

适用角色：所有角色

功能概述

快速开始页面提供两种集成模式的指引：WebSDK 接入和 OpenAPI 接入，帮助您快速完成 OSL Pay 的集成。

使用指南

1. 选择集成模式

页面顶部提供两种集成模式选择：

- **WebSDK 接入**：快速集成，无需后端开发，适合快速上线和测试
- **OpenAPI 接入**：完全控制，适合需要深度定制和后端集成的场景

点击对应的模式按钮即可切换。

2. WebSDK 快速开始（4 步流程）

第一步：访问 Web SDK 测试页面

- 使用测试 appId 访问 Web SDK
- 生产环境：`https://ramp.osl-pay.com/?appId=your_test_appId`
- 沙箱环境：`https://ramptest.osl-pay.com/?appId=your_test_appId`
- 如需获取测试 appId，请联系客户经理

第二步：填写 KYC 信息

- 在沙箱环境中，KYC 信息不会被验证

- 可以上传任意文件进行测试
- 系统不会检查文件的真实性或准确性

第三步：使用测试卡号支付

- 在支付页面选择借记卡/信用卡选项
- 使用提供的测试卡号（不要输入真实卡号）
- 测试卡号列表：
  - Visa 4242 4242 4242 4242（3DS Frictionless）
  - Visa 4539 3732 9896 7400（3DS Challenge）
  - Visa 4532 2274 1657 1592（余额不足，失败）
- 有效期：未来 20 年内任意日期
- CVV：任意 3 位数字

第四步：查看支持的货币和网络

- 沙箱环境支持多种法币和加密货币组合
- EUR → USDT/USDC：最小 1 EUR，最大 43,000 EUR
- EUR → ETH：最小 1 EUR，最大 43,000 EUR
- USD → USDT/USDC：最小 1 USD，最大 53,000 USD
- 支持的网络：ERC20, TRC20, BEP20, Polygon, Arbitrum, Optimism, Solana 等

3. OpenAPI 快速开始（4 步流程）

第一步：获取 API 凭证

- 点击「前往开发者控制台 →」链接
- 查看并复制 App ID、API 公钥和 Webhook 公钥

第二步：实现签名逻辑

- 签名字符串格式：`appId=[your_app_id]&timestamp=[unix_timestamp]`
- 使用 RSA SHA256withRSA 算法对签名字符串进行签名
- 点击「使用签名工具测试 →」链接进行在线测试

第三步：调用 API 接口

- 点击「查看 API 文档 →」链接
- 参考 API 文档，使用您选择的编程语言调用 OSL Pay 的 OpenAPI 接口
- 平台提供多语言代码示例

第四步：配置 Webhook

- 点击「配置 Webhook →」链接

- 配置 Webhook 端点以接收订单状态变更、KYC 状态等实时通知
- 记得验证 Webhook 签名以确保安全

4. 签名与加密指南（OpenAPI 模式）

页面提供详细的签名与加密说明：

- 构造签名字符串：`appId=[your_app_id]&timestamp=[unix_timestamp]`
- 生成签名：使用私钥对签名字符串进行 RSA SHA256withRSA 签名
- 敏感字段加密：使用 Webhook 公钥加密（RSA/ECB/PKCS1Padding）
- 点击「在线测试签名工具」按钮可跳转到签名工具页面

5. 快速链接

页面底部提供三个快速链接卡片：

- **API 文档**：浏览 AI 友好的 API 文档，支持多语言代码生成
- **签名工具**：在线生成和验证签名，测试加密逻辑
- **Webhook 管理**：配置 Webhook 端点，接收实时通知

6. 技术支持

页面底部显示技术支持联系方式：

- 邮箱：support@osl-pay.com
- 官网联系页面：https://www.osl-pay.com/contact-us

仪表盘

适用角色：管理员、商务人员

功能概述

仪表盘提供业务数据的可视化展示，包括交易总额、成功率、订单数、活跃用户等关键指标。

使用指南

1. **查看关键指标**
   - 交易总额：显示累计交易金额
   - 成功率：显示交易成功的百分比
   - 订单数：显示总订单数量
   - 活跃用户：显示活跃用户数量
2. **订单列表**
   - 查看所有订单的详细信息

- 支持按状态筛选（全部、处理中、成功、失败）
- 点击订单可查看详细信息和状态流转历史

3. **数据导出**
   - 点击「📥 导出 CSV」按钮
   - 系统会根据当前筛选条件导出订单数据
   - 下载的 CSV 文件可用 Excel 打开
4. **分页浏览**
   - 使用「上一页」「下一页」按钮翻页
   - 每页显示 20 条订单记录

商户入驻申请

适用角色：管理员、商务人员

功能概述

商户入驻申请模块支持在线提交入驻材料，系统会根据环境（生产/沙箱）展示不同的申请流程。

申请流程

生产环境（4 步流程）：

1. 公司信息
2. 业务信息
3. 文件上传
4. 确认提交

沙箱环境（3 步流程）：

1. 公司信息
2. 业务信息
3. 确认提交（无需上传文件）

使用指南

第一步：填写公司信息

必填字段：

- 公司名称
- 公司地址

- 联系人姓名
- 联系电话
- 联系邮箱

填写完成后点击「下一步」，系统会自动保存为草稿。

第二步：填写业务信息

必填字段：

- 业务类型（从下拉列表选择）
- 月交易量范围
- 支持的法币（如 USD, EUR, HKD）
- 支持的加密货币（如 BTC, ETH, USDT）
- 业务描述

填写完成后点击「下一步」。

第三步：上传文件（仅生产环境）

必须上传以下文件：

- 营业执照（PDF/JPG/PNG，最大 10MB）
- 法人身份证（PDF/JPG/PNG，最大 10MB）
- 银行对账单（PDF/JPG/PNG，最大 10MB）

上传完成后点击「下一步」。

第四步：确认提交

- 检查所有填写的信息
- 确认无误后点击「提交申请」
- 提交后进入申请进展页面

草稿功能

- 每次点击「下一步」时，系统会自动保存当前进度
- 可以随时离开页面，下次访问时会恢复进度
- 页面右上角会显示「草稿」状态标签
- 支持分多次完成申请

申请进展追踪

提交后可查看：

- 申请状态（已提交、审核中、已通过、已拒绝）

- 时间线（提交申请 → 资料审核中 → 审核结果）
- 已填写的公司信息和业务信息
- 已上传的文件列表

演示功能

- 点击「🔄 重新提交（演示用）」按钮
- 可将已提交的申请重置为草稿状态
- 方便多次演示完整流程

成员与权限管理

适用角色：仅管理员

功能概述

成员与权限管理模块允许管理员管理团队成员，分配角色权限，控制不同成员的访问范围。

使用指南

1. 查看团队成员

页面显示当前团队的所有成员列表，包含：

- 成员姓名和邮箱
- 角色（管理员、业务人员、技术开发）
- 状态（已激活、待接受）
- 加入时间

2. 角色说明

页面顶部显示三种角色的权限说明：

- **管理员**：全部权限
- **业务人员**：仅看申请/账单
- **技术开发**：仅看开发者套件

3. 邀请新成员

操作步骤：

1. 点击右上角「邀请成员」按钮
2. 在弹出的模态框中填写：
    - 邮箱地址（必填）

    - 角色权限（从下拉列表选择）
3. 点击「发送邀请」按钮
4. 新成员会出现在列表中，状态为「待接受」
5. 系统会发送邀请邮件到指定邮箱

4. 移除成员

操作步骤：

1. 在成员列表中找到要移除的成员
2. 点击该成员行右侧的「移除」按钮
3. 在确认对话框中点击「确定」
4. 成员从列表中移除

注意事项

- 只有管理员角色可以访问此功能
- 无法移除自己的账号
- 移除成员后，该成员将无法再访问平台
- 邀请邮件的有效期为 7 天

API 文档

适用角色：管理员、技术人员

功能概述

API 文档模块提供 AI 友好的交互式 API 文档，支持场景化 AI 提示词生成、代码示例查看、在线测试等功能。

使用指南

1. 浏览 API 端点

- 左侧列表显示所有 API 端点（共 16 个接口）
- 按分类筛选：
    - 用户管理（2 个接口）
    - 报价（1 个接口）
    - 订单（3 个接口）
    - 银行卡（3 个接口）
    - 配置（3 个接口）

    - 商户（2 个接口）
- 点击端点查看详细信息

2. 查看端点详情

每个端点包含：

- 请求方法和路径
- 请求参数（header、query、path）
- 请求体结构（JSON Schema）
- 响应结构（成功和错误响应）
- 依赖关系（需要先调用的其他 API）
- 错误码列表
- 官方文档链接（点击「查看官方文档」在新标签页打开）

3. AI 提示词功能（重点）

点击以下按钮可复制场景化的 AI 提示词：

- 📋 **生成代码**：生成 API 调用代码
    - 用途：快速生成接入代码
    - 示例：粘贴给 AI，说"请用 TypeScript 实现这个 API"
- 🧪 **生成测试**：生成单元测试代码
    - 用途：自动生成测试用例
    - 示例：粘贴给 AI，说"生成 Jest 测试，覆盖正常和异常场景"
- 🐛 **调试错误**：分析错误原因
    - 用途：排查 API 调用问题
    - 示例：粘贴给 AI，附上错误信息和代码片段
- 🔗 **理解依赖**：解释接口依赖关系
    - 用途：理解 API 调用顺序
    - 示例：粘贴给 AI，问"这个 API 的完整调用流程是什么？"

使用流程：

1. 选择一个 API 端点
2. 点击对应场景的按钮
3. 系统复制 AI 提示词到剪贴板
4. 页面显示「✓ 已复制到剪贴板」
5. 打开 AI 助手（Cursor、Claude、ChatGPT 等）

6. 粘贴提示词，补充具体需求
7. AI 根据提示词生成代码或解答

4. 代码生成

- 切换编程语言（TypeScript、Python、Java、WebSDK）
- 查看不同语言的代码示例
- 代码包含签名逻辑、请求构造、错误处理

5. 错误码查询

- 点击「错误码」标签页
- 查看所有错误码的含义和处理建议
- 支持搜索和筛选

最佳实践

1. **先理解再生成**：先用「理解依赖」了解 API 调用流程，再用「生成代码」
2. **批量复制**：如果需要实现完整业务流程，可依次复制多个端点的提示词
3. **保存常用提示词**：对于经常使用的 API，可保存提示词到项目文档
4. **自定义需求**：在提示词后追加具体要求，如"使用 axios"、"添加 TypeScript 类型"

详细说明请参考：AI_PROMPT_FEATURE.md

MCP 配置中心

适用角色：管理员、技术人员

功能概述

MCP 配置中心帮助您将 OSL Pay API 接入到 AI 编程助手（如 Cursor、Windsurf 等），通过 Model Context Protocol (MCP) 让 AI 助手能够通过远程连接调用 OSL Pay API、自动处理签名逻辑、生成集成代码。

什么是 MCP?

Model Context Protocol (MCP) 是一个开放协议，允许 AI 编程助手通过远程连接调用外部 API。通过配置 MCP，您的 AI 助手可以：

- 自动生成 OSL Pay API 调用代码
- 自动处理 RSA 签名逻辑
- 查询订单状态和报价
- 获取快速开始指南

使用指南

1. 查看当前环境

页面顶部显示当前 API 端点、MCP 端点和环境标识：

- 生产环境：
  - API 端点：`https://openapi.osl-pay.com`
  - MCP 端点：`https://mcp.osl-pay.com/sse`
- 沙箱环境：
  - API 端点：`https://openapitest.osl-pay.com`
  - MCP 端点：`https://mcptest.osl-pay.com/sse`

MCP Server 会根据当前环境自动调用对应的 API 端点。

2. 复制 MCP 配置

操作步骤：

1. 点击「复制配置」按钮
2. 配置会自动复制到剪贴板
3. 配置格式如下：

Code block

```
{
  "mcpServers": {
    "oslpay": {
      "url": "https://mcptest.osl-pay.com/sse",
      "transport": {
        "type": "sse"
      }
    }
  }
}
```

3. 配置 AI 编程助手

根据您使用的工具，打开对应的配置文件：

- **Cursor**：设置 → MCP → 编辑配置文件
- **Windsurf**：设置 → Extensions → MCP
- **其他工具**：查找 `mcp.json` 或 `.mcp/config.json`

将复制的配置粘贴到配置文件中，保存并重启 AI 编程助手。

4. 查看可用工具

页面显示 MCP Server 提供的 6 个工具：

工具名称	功能描述
`oslpay_get_quote`	获取法币到加密货币的报价
`oslpay_create_order`	创建支付订单
`oslpay_query_order`	查询订单状态
`oslpay_generate_signature`	生成 API 请求签名（RSA SHA256withRSA）
`oslpay_get_currency_list`	获取支持的加密货币列表
`oslpay_get_guide`	获取快速开始指南（AI 可读的结构化数据）

5. 开始使用

配置完成后，您可以在 AI 编程助手中使用自然语言与 OSL Pay API 交互：

示例提示词：

- "使用 OSL Pay API 获取 100 USD 到 USDT 的报价"
- "创建一个支付订单，金额 100 USD"
- "查询订单 ORD123456 的状态"
- "生成 API 请求签名"
- "如何集成 OSL Pay API？"（AI 会自动获取快速开始指南）

AI 助手会自动：

1. 通过 SSE 连接到远程 MCP Server
2. 调用对应的 MCP 工具
3. MCP Server 处理 RSA 签名逻辑
4. 添加必要的请求头
5. 返回格式化的结果

技术说明

远程连接：

- MCP Server 部署为远程服务
- AI 助手通过 SSE (Server-Sent Events) 协议连接
- 无需在本地安装任何服务

自动签名处理：

- MCP Server 会自动生成 RSA SHA256withRSA 签名
- 自动添加请求头：`X-App-Id`、`X-Timestamp`、`X-Signature`
- 开发者无需手动处理签名逻辑

环境感知：

- MCP Server 根据配置的 URL 自动识别环境
- 生产环境：`https://mcp.osl-pay.com/sse`
- 沙箱环境：`https://mcptest.osl-pay.com/sse`

常见问题

Q: 为什么 AI 助手无法识别 OSL Pay 工具?

A: 可能原因：

- MCP 配置未正确保存
- AI 助手未重启
- 配置文件路径不正确
- 无法连接到远程 MCP Server

解决方法：

1. 检查配置文件是否正确保存
2. 完全退出并重启 AI 助手
3. 查看 AI 助手的 MCP 日志，确认是否成功连接到远程 Server
4. 检查网络连接，确保可以访问 MCP Server URL

Q: 如何切换环境?

A: 修改配置文件中的 `url` 字段：

- 生产环境：`"url": "https://mcp.osl-pay.com/sse"`
- 沙箱环境：`"url": "https://mcptest.osl-pay.com/sse"`

修改后重启 AI 助手即可。

Q: MCP Server 需要手动安装吗?

A: 不需要。MCP Server 是远程服务，AI 助手通过 SSE 协议连接，无需在本地安装任何服务。

Q: 如何更新 MCP Server?

A: MCP Server 是远程服务，由 OSL Pay 团队维护和更新，无需用户手动更新。

效率提升

使用 MCP 配置后，开发效率显著提升：

操作	传统方式	AI 辅助	效率提升
实现签名	30-60 分钟	0 分钟（自动处理）	∞
理解 API 顺序	15-30 分钟	0 分钟（AI 自动处理）	∞
编写代码	1-2 小时	1-2 分钟（AI 生成）	30-60 倍
调试问题	30-60 分钟	2-5 分钟（AI 定位）	10-15 倍

详细说明请参考：AI_SUPPORT_SUMMARY.md

开发者控制台

适用角色：管理员、技术人员

功能概述

开发者控制台提供 API 密钥管理、环境配置、快速导航等功能。

使用指南

1. 查看 API 凭证

- App ID：应用唯一标识符
- API 端点：API 服务器地址
- API 公钥：用于验证签名的公钥
- Webhook 公钥：用于验证 Webhook 推送的公钥

点击「复制」按钮可快速复制凭证信息。

2. 环境切换

- 点击顶部栏右上角的「生产 / 沙箱」按钮
- 切换后影响所有相关页面（API 端点、入驻流程等）
- 当前环境会以不同颜色标识：
  - 生产环境：红色背景
  - 沙箱环境：绿色背景

3. 沙箱测试指南（沙箱环境下显示）

测试卡号：

- Visa: 4111111111111111（有效期 12/2026，CVV 123）
- Mastercard: 5500000000000004（有效期 12/2026，CVV 123）

支持的货币：

- 法币：USD, EUR, GBP, HKD
- 加密货币：BTC, ETH, USDT, USDC
- 网络：ERC20, TRC20, BTC, Polygon

沙箱限额：

- 单笔交易：$10,000
- 日累计：$50,000

4. 快速链接

- API 文档：跳转到 API 文档页面
- 签名工具：跳转到签名工具页面
- Webhook 管理：跳转到 Webhook 管理页面

域名白名单管理

适用角色：管理员、技术人员

功能概述

域名白名单管理模块允许配置业务域名，用于 Web SDK 来源校验。只有白名单中的域名才能调用 Web SDK。

使用指南

1. 查看已配置域名

页面显示已配置的域名列表，包含：

- 域名地址（包含协议）
- 状态（已激活）
- 添加时间

2. 添加新域名

操作步骤：

1. 在「添加域名」区域的输入框中输入完整域名
2. 域名必须包含协议（http:// 或 https://）
3. 示例：`https://www.example.com`

4. 点击「添加」按钮
5. 系统会验证域名格式
6. 验证通过后，域名添加到列表中

3. 域名格式要求

- 必须包含协议：`http://` 或 `https://`
- 支持子域名：`https://app.example.com`
- 支持端口号：`https://localhost:3000`
- 不支持通配符：不能使用 `*.example.com`

4. 移除域名

操作步骤：

1. 在域名列表中找到要移除的域名
2. 点击该域名行右侧的「移除」按钮
3. 在确认对话框中点击「确定」
4. 域名从列表中移除

5. 用途说明

页面顶部的信息提示框说明：

- 只有白名单中的域名才能调用 Web SDK
- 请添加您的业务域名（包含协议）
- 支持子域名和端口号

常见问题

Q: 为什么添加域名失败?

A: 可能原因：

- 域名格式不正确（缺少 http:// 或 https://）
- 域名已存在于白名单中
- 域名不是有效的 URL

Q: 可以使用通配符吗?

A: 不支持通配符。如果需要多个子域名，请分别添加：

- `https://www.example.com`
- `https://app.example.com`
- `https://api.example.com`

Q: 本地开发环境可以添加吗?

A: 可以。支持添加 localhost 和 IP 地址:

- `http://localhost:3000`
- `http://127.0.0.1:8080`
- `http://192.168.1.100:3000`

签名工具

适用角色: 管理员、技术人员

功能概述

签名工具提供在线签名生成、验证和加密功能，帮助开发者快速调试签名逻辑。

使用指南

1. 生成签名

页面已预填测试数据，可直接使用:

- App ID: `demo_app_20240101`
- Timestamp: 当前 Unix 时间戳
- 私钥: 测试用的 RSA 私钥（PKCS#8 格式）

操作步骤:

1. 点击「生成签名」按钮
2. 查看签名字符串格式: `{timestamp}\n{appId}`
3. 查看 Base64 编码的签名结果
4. 查看请求头格式: `open-api-sign: {signature}`

2. 验证签名

生成签名后，切换到「验证签名」标签页:

- 系统会自动填入上一步生成的参数
- App ID、Timestamp、签名值会自动传递
- 公钥已预填测试数据

操作步骤:

1. 确认参数已自动填入
2. 点击「验证签名」按钮
3. 查看验证结果: 「✓ 签名验证通过」或「✗ 签名验证失败」

3. 加密数据

切换到「加密数据」标签页：

- 输入明文（默认：Hello OSLpay!）
- 公钥已预填测试数据

操作步骤：

1. 输入要加密的明文
2. 点击「加密」按钮
3. 查看 Base64 编码的密文
4. 点击「复制」按钮复制密文

4. 重置测试数据

点击「🔄 重置为测试数据」按钮，可恢复所有预填的测试数据。

技术说明

- 签名算法：RSA SHA256withRSA
- 加密算法：RSA/ECB/PKCS1Padding
- 密钥格式：PKCS#8 PEM 格式
- 签名字符串格式：`{timestamp}\n{appId}`

常见问题

Q: 为什么验证签名失败?

A: 可能原因：

- 私钥和公钥不匹配
- 签名字符串格式不正确
- 密钥格式不正确（需要 PKCS#8 PEM 格式）

Q: 如何生成自己的 RSA 密钥对?

A: 使用 OpenSSL 命令：

Code block

```
# 生成私钥
openssl genrsa -out private.pem 2048

# 转换为 PKCS#8 格式
openssl pkcs8 -topk8 -inform PEM -in private.pem -outform PEM -nocrypt -out private_pkcs8.pem

```

```
# 生成公钥
openssl rsa -in private.pem -pubout -out public.pem
```

Webhook 管理

适用角色：管理员、技术人员

功能概述

Webhook 管理模块允许配置事件推送，当系统中发生特定事件时，自动向指定的 URL 发送 HTTP POST 请求。

使用指南

1. 创建 Webhook

操作步骤：

1. 点击右上角「+ 新建 Webhook」按钮
2. 填写目标 URL（必须是 HTTPS 地址）
3. 选择要订阅的事件类型（可多选）：
   - 订单事件：order.created, order.completed, order.failed
   - 支付事件：payment.success, payment.refund
   - KYC 事件：kyc.approved, kyc.rejected
4. 点击「保存」
5. 系统自动生成签名密钥（Secret）

2. 编辑 Webhook

操作步骤：

1. 在配置列表中点击「编辑」按钮
2. 修改 URL 或订阅的事件类型
3. 点击「保存」

3. 测试 Webhook

操作步骤：

1. 在配置列表中点击「测试」按钮
2. 系统向配置的 URL 发送测试消息
3. 在右侧推送日志中查看测试结果

4. 查看推送日志

操作步骤：

1. 在左侧配置列表中点击任意 Webhook
2. 右侧面板显示该 Webhook 的所有推送日志
3. 日志包含：
   - 事件类型
   - 推送状态（成功/失败/重试中）
   - HTTP 响应状态码
   - 重试次数
   - 推送时间

5. 删除 Webhook

操作步骤：

1. 在配置列表中点击「删除」按钮
2. 确认删除
3. 相关的推送日志也会被清理

Webhook 推送机制

请求格式：

HTTP Headers:

Code block

```
Content-Type: application/json
X-Webhook-Secret: {签名密钥}
X-Webhook-Event: {事件类型}
```

Request Body:

Code block

```
{
  "event": "order.created",
  "timestamp": "2024-03-03T10:30:00Z",
  "data": {
    "orderId": "ORD20240303001",
    "amount": 100.00,
    "currency": "USD",
    "status": "pending"
```

```
  }
}
```

重试策略：

- 如果推送失败，系统会自动重试
- 采用指数退避策略：1秒、2秒、4秒、8秒、16秒
- 最多重试 5 次
- 5 次后仍失败，状态变为「final_failed」

推送状态：

- `pending` - 等待推送
- `success` - 推送成功（收到 2xx 响应）
- `retry_pending` - 等待重试
- `final_failed` - 最终失败

接收端实现示例

Node.js 示例：

Code block

```
const express = require('express');
const app = express();

app.post('/webhook', express.json(), (req, res) => {
  // 1. 验证签名密钥
  const receivedSecret = req.headers['x-webhook-secret'];
  const expectedSecret = process.env.WEBHOOK_SECRET;

  if (receivedSecret !== expectedSecret) {
    return res.status(401).send('Unauthorized');
  }

  // 2. 获取事件类型
  const eventType = req.headers['x-webhook-event'];

  // 3. 处理事件数据
  const event = req.body;
  console.log(`Received ${eventType}:`, event);

  // 4. 快速返回 200（避免超时）
  res.status(200).send('OK');

```

```
  // 5. 异步处理业务逻辑
  processEventAsync(event).catch(err => {
    console.error('Failed to process webhook:', err);
  });
});

app.listen(3000);
```

安全建议

1. **验证签名密钥**：接收端必须验证 `X-Webhook-Secret` 请求头
2. **实现幂等性**：同一事件可能因重试而多次推送，需要去重处理
3. **快速响应**：接收端应在 5 秒内返回 2xx 响应
4. **异步处理**：将耗时操作放到后台队列处理
5. **使用 HTTPS**：生产环境必须使用 HTTPS URL

常见问题

Q: 为什么 Webhook 一直显示「retry_pending」？

A: 可能原因：

- 目标 URL 无法访问或响应超时
- 服务器返回非 2xx 状态码
- 防火墙阻止了来自 OSLpay 的请求

解决方法：

- 使用「测试」功能验证 URL 是否可访问
- 检查服务器日志，确认是否收到请求
- 确保服务器返回 200-299 状态码

Q: 如何知道推送失败的原因?

A: 点击配置查看推送日志，日志中包含：

- HTTP 响应状态码
- 响应内容（如果有）
- 错误信息（如连接超时、DNS 解析失败等）

详细说明请参考：WEBHOOK_FEATURE.md

API 请求日志

适用角色：管理员、技术人员

功能概述

API 请求日志模块展示最新 10 条 API 请求流水，用于排查集成错误。日志包含完整的 Request 和 Response 内容。

使用指南

1. 查看日志列表

页面显示最新 10 条 API 请求记录，包含：

- 时间戳（精确到秒）
- HTTP 方法（GET/POST/PUT/DELETE）
- 请求路径
- 状态码
- 响应耗时（毫秒）

2. HTTP 方法颜色标识

不同的 HTTP 方法使用不同颜色的标签：

- GET：绿色标签
- POST：蓝色标签
- PUT：黄色标签
- DELETE：红色标签

3. 状态码颜色标识

不同的状态码使用不同颜色显示：

- 2xx（成功）：绿色
- 4xx（客户端错误）：黄色
- 5xx（服务器错误）：红色

4. 查看详细信息

操作步骤：

1. 在日志列表中点击任意记录
2. 或点击记录右侧的「查看详情」按钮
3. 系统打开详情模态框，显示：
    - 请求信息（方法、路径、状态码、耗时）
    - Request Body（JSON 格式化显示）

    - Response Body（JSON 格式化显示）
4. 点击右上角的「×」或「关闭」按钮关闭模态框

5. 日志刷新

页面顶部的信息提示框说明：

- 日志每 5 秒自动刷新
- 仅保留最新 10 条记录
- 点击任意记录查看详细的 Request/Response 内容

使用场景

场景 1：排查 API 调用失败

1. 在日志列表中找到失败的请求（状态码 4xx 或 5xx）
2. 点击查看详情
3. 检查 Request Body 是否正确
4. 查看 Response Body 中的错误信息
5. 根据错误码和错误消息定位问题

场景 2：验证签名是否正确

1. 调用 API 后，在日志中找到对应的请求
2. 查看状态码：
    - 401：签名验证失败
    - 200：签名验证成功
3. 如果失败，检查签名字符串格式和签名算法

场景 3：检查请求参数

1. 在日志中找到对应的请求
2. 查看 Request Body
3. 确认所有必填参数都已提供
4. 确认参数格式和类型正确

注意事项

- 日志仅保留最新 10 条，旧记录会被自动清理
- 敏感信息（如密钥、密码）不会记录在日志中
- 日志数据仅用于调试，不作为审计依据
- 生产环境和沙箱环境的日志是独立的

常见问题

Q: 为什么看不到我的请求?

A: 可能原因:

- 请求还未到达服务器
- 请求被防火墙拦截
- 日志已被新的请求覆盖（仅保留最新 10 条）

Q: 如何导出日志?

A: 当前版本不支持导出功能。如需保存日志，请手动复制 Request/Response 内容。

Q: 日志会保留多久?

A: 日志仅保留最新 10 条记录，没有时间限制。当有新请求时，最旧的记录会被自动清理。

常见问题

登录与权限

Q1: 忘记密码怎么办?

A: 请联系系统管理员重置密码，或使用「忘记密码」功能（如果已启用）。

Q2: 为什么我看不到某些菜单?

A: 不同角色有不同的权限:

- 商务人员只能看到仪表盘和商户入驻申请
- 技术人员只能看到 API 文档、开发者控制台、签名工具、Webhook 管理
- 管理员可以看到所有菜单

Q3: 如何切换角色?

A: 角色由系统管理员分配，无法自行切换。如需更改角色，请联系管理员。

商户入驻

Q4: 草稿会保存多久?

A: 草稿会永久保存，直到您提交申请或删除草稿。

Q5: 提交后可以修改吗?

A: 提交后无法直接修改。如需修改，请联系审核人员或使用「重新提交」功能（演示环境）。

Q6: 沙箱环境和生产环境的区别?

A:

- 沙箱环境：3 步流程，无需上传文件，审核简化，用于测试
- 生产环境：4 步流程，需要上传 3 个文件，完整审核，用于正式业务

API 文档

Q7: 复制 AI 提示词后粘贴板是空的?

A: 浏览器的剪贴板 API 需要 HTTPS 或 localhost 环境。确保在 `https://` 或 `http://localhost` 访问。

Q8: AI 生成的代码不符合需求怎么办?

A: 在提示词后追加具体要求，例如：

- "请使用 async/await 而不是 Promise.then"
- "请添加详细的注释"
- "请使用我们的自定义 HTTP 客户端"

Q9: 提示词太长，AI 无法处理怎么办?

A: 只复制需要的部分，或分多次提问：

1. 第一次：复制提示词，问"这个 API 的核心参数是什么？"
2. 第二次：问"帮我生成调用代码"
3. 第三次：问"如何处理错误？"

签名工具

Q10: 为什么验证签名失败?

A: 可能原因：

- 私钥和公钥不匹配
- 签名字符串格式不正确（应该是 `{timestamp}\n{appId}`）
- 密钥格式不正确（需要 PKCS#8 PEM 格式）

Q11: 如何生成自己的 RSA 密钥对?

A: 使用 OpenSSL 命令：

Code block

```
# 生成私钥
openssl genrsa -out private.pem 2048

# 转换为 PKCS#8 格式
openssl pkcs8 -topk8 -inform PEM -in private.pem -outform PEM -nocrypt -out private_pkcs8.pem

```

```
# 生成公钥
openssl rsa -in private.pem -pubout -out public.pem
```

Webhook

Q12: Webhook 一直显示「retry_pending」？

A: 可能原因：

- 目标 URL 无法访问或响应超时
- 服务器返回非 2xx 状态码
- 防火墙阻止了来自 OSLpay 的请求

解决方法：

- 使用「测试」功能验证 URL 是否可访问
- 检查服务器日志，确认是否收到请求
- 确保服务器返回 200-299 状态码

Q13: 如何防止重复处理同一事件？

A: 实现幂等性处理：

Code block

```
const processedEvents = new Set();

app.post('/webhook', (req, res) => {
  const eventId = req.body.id || req.body.timestamp;

  if (processedEvents.has(eventId)) {
    return res.status(200).send('Already processed');
  }

  processEvent(req.body);
  processedEvents.add(eventId);
  res.status(200).send('OK');
});
```

环境切换

Q14: 切换环境后数据会丢失吗？

A: 不会。生产环境和沙箱环境的数据是独立的，切换环境只是改变显示的数据源。

Q15: 如何知道当前在哪个环境？

A: 查看顶部栏右上角的环境标识：

- 生产环境：红色背景
- 沙箱环境：绿色背景

mike.xu 2026-03-18 07:46 UTC